

MIDTERM PRESENTATION

ARTIFICIAL INTELLIGENCE IN SOKOBAN

**Course: Introduction to Artificial
Intelligence Code: 503043
Group: 09**

Students list

	Student ID	Student Name	Tasks	Contribution
1	524H0135	Nguyễn Phan Anh Tuấn	1 + 2	25%
2	524H0137	Đỗ Quốc Việt	3 + 4	25%
3	524H0132	Đỗ Quốc Trung	5 + 6	25%
4	524H0134	Hà Dư Tuấn	7 + 8	25%

Task 1 - STATE-SPACE SEARCH FORMULATION



```
// 1. Data Segregation
Static_Environment = Set(Walls), Set(Targets)
Current_State = (Agent_Position, Sorted_Tuple(Boxes_Position))

// 2. Transition Model (Action Logic)
function Get_Successors(Current_State):
    for each direction in [North, South, East, West]:
        if next_cell == Wall:
            continue // Blocked

        if next_cell == Box:
            if cell_behind_box == Wall or Box:
                continue // Cannot push
            Push_Box()

    Update_Agent_Position()
    return New_State
```

1. State Space (S): (See Data Segregation section)

- *Static:* Walls & Targets in `Set()` → $O(1)$ lookup.
- *Dynamic:* Agent & Boxes in sorted `Tuple` → prevents unhashable memory errors.

2. Initial State (S_0): Parsed from map files via (x, y) coordinates.

3. Actions (A): $A = \{ \text{North, South, East, West} \}$.

4. Transition Model $T(s, a)$: (See Action Logic section)

- Validates physical constraints (e.g., wall blocking, push mechanics).

5. Goal Test (G): Uses tuple unpacking (`_, boxes = state`) to exclusively compare box coordinates with targets.

Task 2 - SEARCH ALGORITHMS & HEURISTIC DESIGN



```
function A_Star_Search(Start_State):
    PriorityQueue = []
    visited = Set()
    counter = 0 // Tie-breaker ID

    // Tuple format: (F_cost, G_cost, Counter, State, Path)
    Push PriorityQueue(start_h, 0, counter, Start_State, [])

    while PriorityQueue is not empty:
        // Use tuple unpacking to ignore counter (_)
        pop(f_cost, g_cost, _, current_state, path)

        if current_state is Goal:
            return path

        if current_state not in visited:
            add current_state to visited

            for each Successor in Get_Successors(current_state):
                counter += 1
                Push PriorityQueue(new_f, new_g, counter, next_state, new_path)
```

Algorithm

Implementations: Deployed both Uniform Cost Search (UCS) for exhaustive pathfinding and A* Search for heuristic-driven optimization.

Priority Queue Stability:

Injected a sequential **counter** variable into the **heapq** tuple. This acts as a strict tie-breaker, preventing fatal system crashes caused by raw tuple comparison when two states share identical costs.

Task 2 - SEARCH ALGORITHMS & HEURISTIC DESIGN



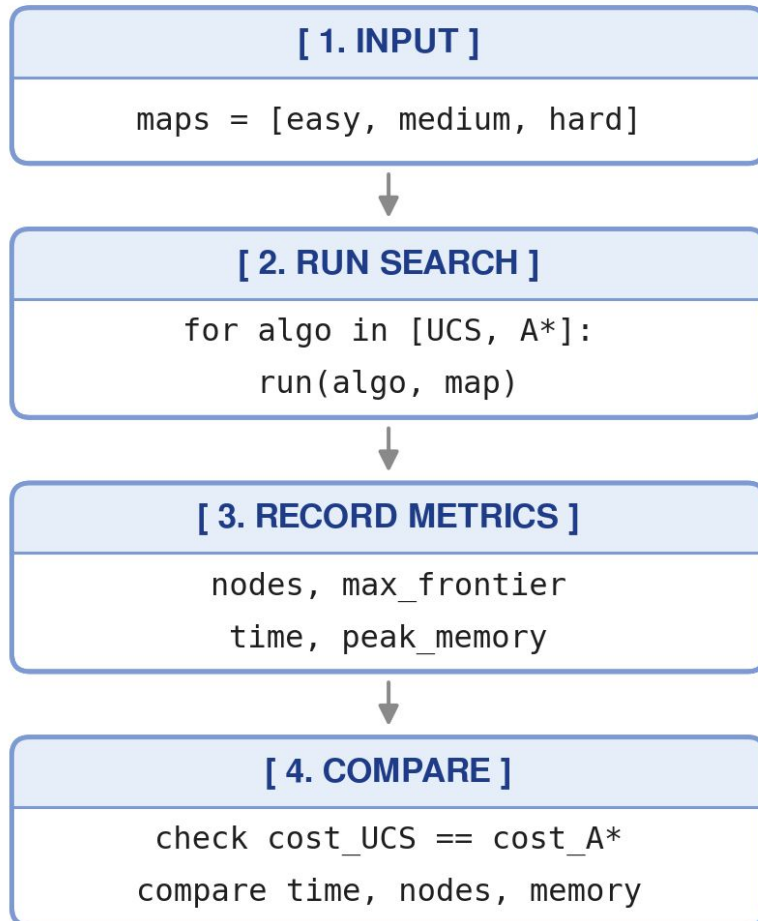
```
function Calculate_Heuristic(Boxes, Targets):  
    total_cost = 0  
    for each Box in Boxes:  
        if Box is on Target: continue  
  
        // 1. DEADLOCK PRUNING  
        if Check_Corner_Deadlock(Box) == True:  
            return INFINITY // Branch terminated  
  
        // 2. ADMISSIBLE ESTIMATION (Chebyshev)  
        min_dist = MAX(abs(Box.x - Target.x), abs(Box.y -  
Target.y))  
        total_cost += min_dist  
  
    return total_cost
```

Admissible Estimation:

Substituted the prohibited metrics with **Chebyshev Distance**. Since diagonal moves are illegal, it always strictly underestimates the true movement cost in Sokoban.

Deadlock Pruning: Introduced an `is_deadlock()` geometric check. If a box is trapped in a 90-degree corner, the heuristic forces $h(n) = \infty$. This triggers aggressive **Pruning**, permanently discarding fatal branches and dramatically accelerating execution time.

Task 3: Experiment Design (UCS VS. A*)



Goal:

- Compare the time and space cost of UCS and A* on the same Sokoban problem.

Test maps:

- 3 levels: easy, medium, hard (the given example map).

Metrics:

- Running time (average of 3 runs).
- Expanded nodes and maximum frontier size.
- Peak memory (measured in a separate run).

Fair comparison:

- Same problem model and tie-breaking.
- Only the priority differs: g (UCS) vs. $g + h$ (A*).

Task 3: Test Maps

<i>map_easy.txt</i>
%%%%%%%%%%
%D D %
% B B A %
% %
%%%%%%%%%%

EASY

2 boxes

<i>map_medium.txt</i>
%%%%%%%%%%
%D D D %
% B B B %
% A %
% %% %
%%%%%%%%%%

MEDIUM

3 boxes

<i>example_map.txt</i>
%%%%%%%%
%%% %
%DAB %
%%% BD%
%D%B %
% % D %%
%B CBB%
% D %
%%%%%%%%

HARD

7 boxes (given example map)

% = wall

A = agent

B = box

D = target

C = box on target

Task 3: Results

Map	Algorithm	Expanded nodes	Max frontier	Time	Peak memory
map_easy	UCS	509	315	0.002 s	87 KB
	A*	201	139	0.002 s	28 KB
map_medium	UCS	32211	17437	0.30 s	13.0 MB
	A*	8958	5100	0.19 s	3.3 MB
example_map	UCS	485182	145563	8.33 s	162 MB
	A*	14553	5613	0.54 s	5.1 MB

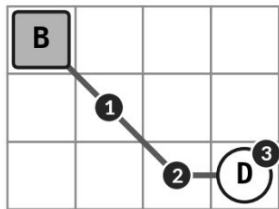
Task 4: Heuristic Properties

[1. ADMISSIBLE]

$$h(s) \leq h^*(s)$$

h never overestimates the true cost

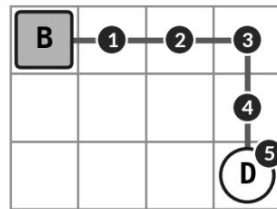
Example: 1 box (B), 1 target (D)



$h = 3$

3 steps (diagonal OK)

$\leq$



real cost ≥ 5

5 pushes (no diagonal)

Why: A* returns an optimal solution

[2. CONSISTENT]

$$h(s) \leq c(s, s') + h(s')$$

one step lowers h by at most 1

Example: push the box 1 cell (cost 1)



↓ push right, cost 1



$$3 \leq 1 + 2 \quad \checkmark$$

If only the agent walks, h stays 3: $3 \leq 1 + 3 \quad \checkmark$

Why: A* can skip visited nodes safely

Task 4: Heuristic Admissibility and Consistency Check

Goal:

- Check that the heuristic is admissible (never overestimates) and consistent, so A* stays optimal.

Test maps:

- Small: `map_easy`.
- Medium: `map_medium`.
- Large: `example_map`.

Coverage:

- Small and medium maps: every reachable state is checked.
- Large maps: too many states, so only the UCS optimal path and the first N states of the BFS are checked.

Criterion:

- Zero violations means the heuristic is valid.

[1. INPUT]

```
exhaustive = [map_easy, map_medium]
sampled     = [example_map]
```



[2. EXHAUSTIVE: small maps]

```
states = BFS(start) # all states
h* = backward_BFS(goal_states)
h = calculate_heuristic(s)
```



[3. SAMPLED: large maps]

```
path, C* = UCS(map)
h*(s_i) = C* - i # on optimal path
edges = BFS(start, N states)
```



[4. CHECK]

```
admissible: h(s) <= h*(s)
consistent: h(s) <= c + h(s')
deadlock:   h = inf => h* = inf
```



[5. OUTPUT]

```
count violations, print report
no violation => admissible & consistent
save heuristic_verification.txt
```

Task 4: Results

Map	Method	States	Admissible violations	Consistency violations
map_easy	Exhaustive	12176	0	0
map_medium	Exhaustive	294989	0	0
example_map	Sampling	100000	0	0

Exhaustive: all reachable states, exact h^* by backward BFS from the goals.

Sampling: UCS optimal path (cost 34) and the first 100,000 BFS states.

Task 5 - INITIAL STATE & ALGORITHM SELECTOR OVERLAY



UX & Visual Focus

- Semi-transparent Overlay: Use `pygame.Surface` with an integrated Alpha channel (150) to create a semi-transparent overlay over the map, drawing visual attention to the main content.
- Direct Key Binding: Listen for key events from the 1 (UCS) and 2 (A)* keys on both the main keyboard and the Numpad.

State Management & Control

- State Flag Control: Initialize `selected_algo = "None"` with a status label indicating WAITING.
- State-based Guarding: Use Guard Clauses to lock keyboard inputs (Space, ←/→), preventing invalid operations before a path has been generated.

Visibility of System Status

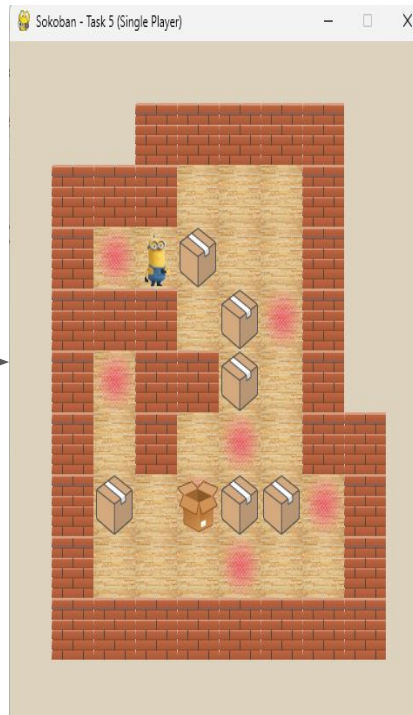
- Visual State Feedback: Clearly display the system status through a WAITING badge and an instruction message below the Bottom Bar.

Task 5 - OOP ARCHITECTURE & MAP MATRIX PARSING

example_map.txt

```

#####
##### 
%DAB  %
##### BD%
%D%B %
% % D %
%B CBB%D
%   D  %
#####
  
```



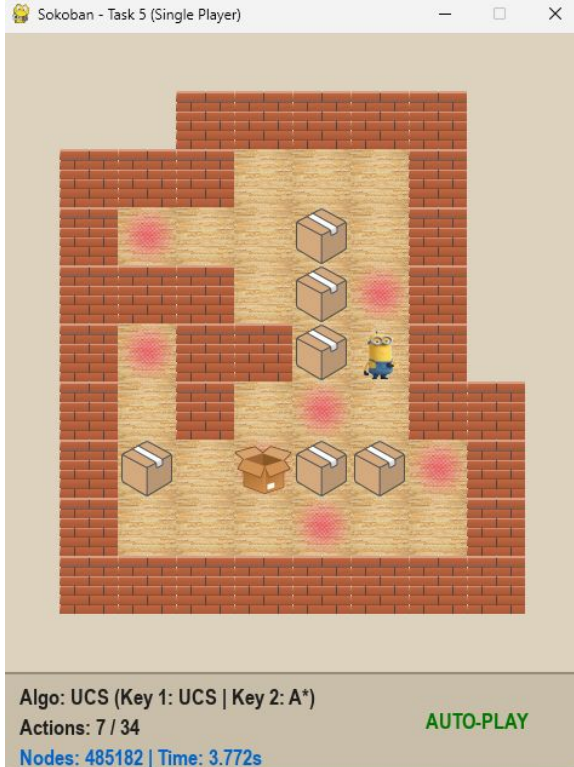
Map Matrix Parsing (Board Class)

- **Symbol Mapping:** Reads the .txt file and maps symbols: % (Wall), (Floor), D (Goal), B (Box), and A (Agent).
- **State Separation:** Separates the fixed static matrix (% , D) from the dynamic coordinate data (agent_pos, boxes_pos).
- **Collision Validation:** The `is_valid_move()` function checks physical collision rules (preventing movement through walls and preventing the pushing of two consecutive boxes).

Sprite Graphics Optimization

Seamless Sprite Cropping: Uses `get_bounding_rect()` and `subsurface()` to remove excess transparent borders from the original images, ensuring that wall and floor tiles fit together seamlessly without visible gaps.

Task 5 - REAL-TIME EXECUTION & UX CONTROLS



Non-blocking Execution Loop

- Smooth Movement Timing: Uses `pygame.time.get_ticks()` to introduce a 300ms delay between movement steps, allowing the character to move smoothly without freezing the screen.
- Pause & Resume Control: The Space key allows users to pause or resume the animation at any time.

Step Playback Mechanism

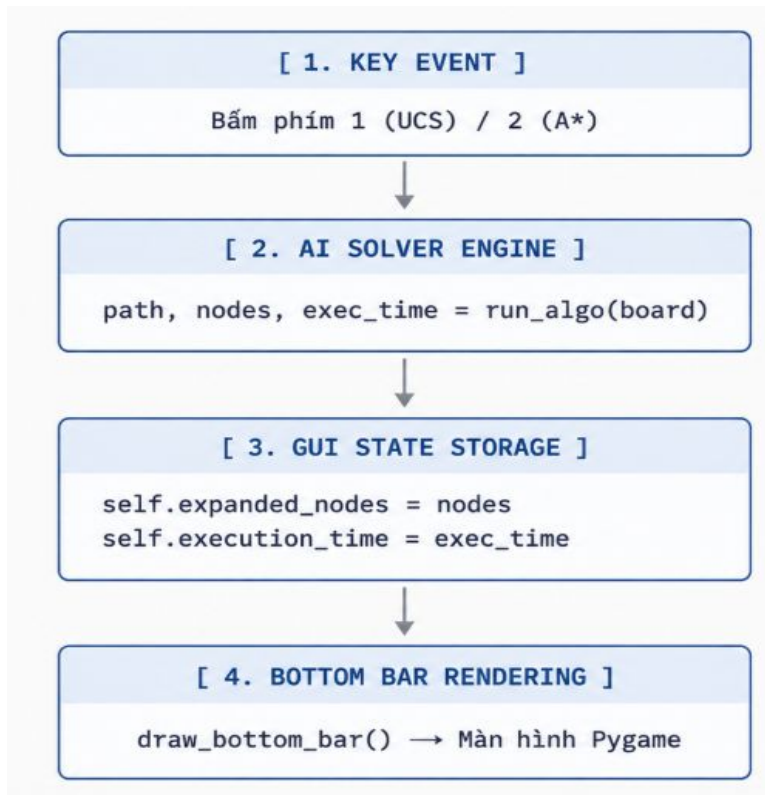
Step-by-step Navigation: The $\leftarrow$ / $\rightarrow$ arrow keys allow users to instantly review each movement step on the screen (rewind / fast-forward).

Real-time Metrics Dashboard

Technical Metrics Display: The Bottom Bar directly displays key performance metrics:

- Actions: Number of movement steps.
- Expanded Nodes: Number of states explored by the algorithm.
- Execution Time: Time taken by the algorithm to find the solution.

TASK 5: ALGORITHM-TO-GUI DATA INTEGRATION PIPELINE



- **Event Dispatcher:** Detects the key press 1 (UCS) or 2 (A*), triggering the corresponding search algorithm.
- **Tuple Unpacking:** Receives the returned data as a three-element tuple: (solution_path, expanded_nodes, execution_time).
- **State Binding:** Stores the values calculated by the AI Solver in internal instance attributes: `self.expanded_nodes` and `self.execution_time`.
- **Dynamic Rendering:** The `draw_bottom_bar()` function reads state variables → Converts them into text strings → Renders directly on the bottom bar in real time.

TASK 6: COMPETITIVE 2-AGENT ENVIRONMENT & RULES

Symbol Legend	
%1	2%
%D % % D%	% : Wall – Impassable.
% B B %	(space) : Floor – Valid movement area.
%D% B %D%	B : Box – Pushable object (5 boxes in total).
% B B %	D : Goal – Target location to push boxes into (6 goals in total).
%D % % D%	1 : Starting position of Agent 1 (Left).
% %	2 : Starting position of Agent 2 (Right).

**Simultaneous
Real-time**

Dynamic Stealing

**Collision &
Obstacle**

Step-limit Victory

Time Constraint: Maximum timeout of 1000 ms per Agent decision step.

TASK 6: MAPF FORMULATION & DECISION CONSTRAINTS



PROBLEM FORMULATION

- ✓ State S: (p1, p2, B)
- ✓ Goal: Box B -> Goal D
- ✓ Independent Engine



2. SEARCH SPACE BOUNDS

- Static: Wall Avoidance
- Dynamic: Opponent = Wall
- Action: Single Box Push

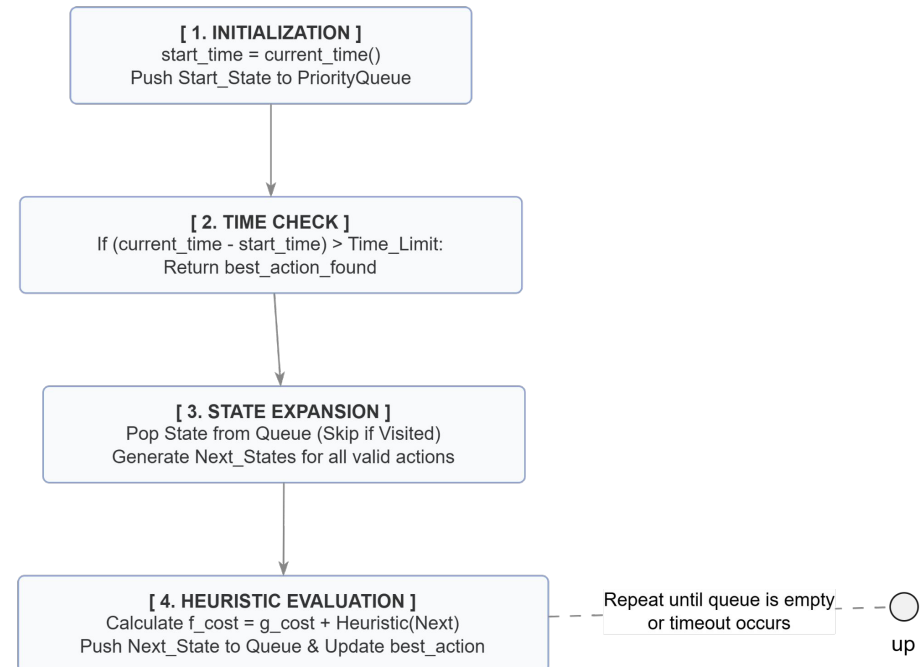


3. REAL-TIME EXECUTION

- Timeout: $\leq 1000\text{ms/step}$
- Failsafe: Depth Cutoff
- Independent Script File

TASK 7: AGENT CONTROL ALGORITHMS

- **AI Strategies:** A* (Agent 1) and GBFS (Agent 2)
- **Heuristic Design:** Calculates score based on box ownership (+2000 for claiming, -2000 if robbed), Manhattan distances to goal and includes deadlock detection to prune invalid states.
- **Time Limit Handling:** Uses ThreadPoolExecutor to run calculations asynchronously. If the safe limit exceeded, the agent immediately returns the best fallback action.



TASK 8: COMPETITIVE GAMEPLAY IMPLEMENTATION

- **Game Rules:** 2 Agents compete simultaneously within a maximum number of steps, both process the game state and perform their actions concurrently per turn
- **Mechanism:** Agent cannot pass through each other, but able to push a box out of the opponent's designated goal to claim ownership.
- **Conflict Resolution:**
 - Actions are executed and resolved **simultaneously**
 - If both agents attempt to move into the **same cell**, push the **same box** or **collide head-on**, moves are **canceled**

